DGP Scenario Reproduction

Arjun Sekhar

2026-06-14

Chunk 1: Libraries

```
library(randomForestSRC)
library(survival)
library(PRROC)
library(pander)
```

Chunk 2: Function definitions

```
set.seed(44940076)

## Step 1: Data generating process
generate_survival_data <- function(n, p = 10, betas,
                                   interactions = NULL,
                                   censoring_rate = 0.99) {

  ## Step 1a: Generate orthogonal covariates (mimics PCA structure of ULB V1-V28)
  X <- matrix(rnorm(n * p), nrow = n, ncol = p)
  colnames(X) <- paste0("X", 1:p)

  ## Step 1b: Compute linear predictor
  lp <- X %*% betas

  ## Step 1c: Add interaction terms if specified (Scenario B only)
  if (!is.null(interactions)) {
    for (term in interactions) {
      lp <- lp + term$gamma * X[, term$i] * X[, term$j]
    }
  }

  ## Step 1d: Generate exponential survival times
  lambda     <- exp(lp)
  time_event <- rexp(n, rate = lambda)

  ## Step 1e: Apply censoring to hit target censoring rate
  lambda_c   <- quantile(time_event, probs = 1 - censoring_rate)
  time_censor <- rexp(n, rate = 1 / as.numeric(lambda_c))

  ## Step 1f: Observed time and event indicator
  time_obs <- pmin(time_event, time_censor)
  status   <- as.integer(time_event <= time_censor)
```

```r
  data.frame(X, time = time_obs, status = status)
}

## Step 2: Scenario runner
run_scenario <- function(scenario_label, n, betas,
                         interactions = NULL,
                         censoring_rate,
                         n_splits = 30) {

  auprc_lr  <- numeric(n_splits)
  auprc_rsf <- numeric(n_splits)
  cerr_lr   <- numeric(n_splits)
  cerr_rsf  <- numeric(n_splits)
  n_events  <- numeric(n_splits)

  for (i in seq_len(n_splits)) {
    message("  [", scenario_label, "] Split ", i, " of ", n_splits,
            " - ", Sys.time())

    ## Step 2a: Generate data from known DGP
    dat <- generate_survival_data(n, betas = betas,
                                  interactions = interactions,
                                  censoring_rate = censoring_rate)

    n_events[i] <- sum(dat$status)

    ## Step 2b: Stratified 70/30 train-test split
    event_idx    <- which(dat$status == 1)
    nonevent_idx <- which(dat$status == 0)

    train_idx <- c(
      sample(event_idx,    size = floor(0.7 * length(event_idx))),
      sample(nonevent_idx, size = floor(0.7 * length(nonevent_idx)))
    )

    train <- dat[train_idx, ]
    test  <- dat[-train_idx, ]

    ## Step 2c: Logistic Regression
    lr_mod  <- glm(status ~ . - time, data = train, family = binomial)
    lr_prob <- predict(lr_mod, newdata = test, type = "response")

    ## Step 2d: Random Survival Forest
    rsf_mod  <- rfsrc(Surv(time, status) ~ ., data = train,
                      ntree = 100, nodesize = 15,
                      splitrule = "logrank", fast = TRUE, nthread = 4)

    rsf_pred <- predict(rsf_mod, newdata = test[, rsf_mod$xvar.names,
                                                drop = FALSE])
    rsf_risk <- 1 - rsf_pred$survival[, ncol(rsf_pred$survival)]
    rsf_risk <- as.numeric(rsf_risk)

    true_labels <- test$status
```

```
    ## Step 2e: AUPRC
    lr_keep  <- is.finite(lr_prob)  & is.finite(true_labels)
    rsf_keep <- is.finite(rsf_risk) & is.finite(true_labels)

    auprc_lr[i] <- pr.curve(
      scores.class0 = lr_prob[lr_keep][true_labels[lr_keep] == 1],
      scores.class1 = lr_prob[lr_keep][true_labels[lr_keep] == 0],
      curve = FALSE)$auc.integral

    auprc_rsf[i] <- pr.curve(
      scores.class0 = rsf_risk[rsf_keep][true_labels[rsf_keep] == 1],
      scores.class1 = rsf_risk[rsf_keep][true_labels[rsf_keep] == 0],
      curve = FALSE)$auc.integral

    ## Step 2f: Concordance error (1 - C-index)
    ## Note: concordance() uses the survival convention where higher predictor =
    ## longer survival. Since lr_prob and rsf_risk are RISK scores (higher = sooner
    ## failure), the raw concordance returns < 0.5. CErr = 1 - concordance()
    ## therefore recovers the true discriminating C-index (> 0.5 = good).
    ## This must be stated explicitly when reporting these values in the thesis.
    cerr_lr[i]  <- 1 - as.numeric(concordance(
      Surv(test$time, test$status) ~ lr_prob)$concordance)
    cerr_rsf[i] <- 1 - as.numeric(concordance(
      Surv(test$time, test$status) ~ rsf_risk)$concordance)

    ## Step 2g: Clean up
    rm(rsf_mod, rsf_pred, dat, train, test)
    gc()
  }

  data.frame(
    Scenario     = scenario_label,
    AUPRC_LR     = mean(auprc_lr),   AUPRC_LR_SD  = sd(auprc_lr),
    AUPRC_RSF    = mean(auprc_rsf),  AUPRC_RSF_SD = sd(auprc_rsf),
    CErr_LR      = mean(cerr_lr),    CErr_LR_SD   = sd(cerr_lr),
    CErr_RSF     = mean(cerr_rsf),   CErr_RSF_SD  = sd(cerr_rsf),
    Events_mean  = round(mean(n_events)),
    Events_SD    = round(sd(n_events))
  )
}

betas_A <- c(0.5, 0.4, 0.3, 0.3, 0.2, 0.2, 0.1, 0.1, 0.05, 0.05)
```

Note: concordance() uses the survival convention where higher predictor = longer survival. Since lr_prob and rsf_risk are RISK scores (higher = sooner failure), the raw concordance returns < 0.5. CErr = 1 - concordance() therefore recovers the true discriminating C-index (> 0.5 = good).

I will need to state this explicitly when reporting these values in the thesis.

Chunk 3: Run

```
## Step 3: Execute scenarios
## Delete dgp_scenario_results.rds to re-run
```

```r
if (!file.exists("dgp_scenario_results.rds")) {
  results_A <- run_scenario(
    scenario_label = "A: Linear additive, high censoring",
    n              = 10000,
    betas          = betas_A,
    interactions   = NULL,
    censoring_rate = 0.99
  )
  results_B <- run_scenario(
    scenario_label = "B: Interactions, moderate censoring",
    n              = 5000,
    betas          = betas_A,
    interactions   = list(
      list(i = 1, j = 3, gamma = 0.5),
      list(i = 2, j = 4, gamma = 0.5)
    ),
    censoring_rate = 0.50
  )
  results_combined <- rbind(results_A, results_B)
  saveRDS(results_combined, "dgp_scenario_results.rds")
} else {
  results_combined <- readRDS("dgp_scenario_results.rds")
}
```

Chunk 4: Display

```r
pander::pander(results_combined, digits = 4,
       caption = "Table: DGP Scenario Reproduction - LR vs RSF under controlled conditions")
```

Table 1: Table: DGP Scenario Reproduction — LR vs RSF under controlled conditions (continued below)

Scenario	AUPRC_LR	AUPRC_LR_SD	AUPRC_RSF	AUPRC_RSF_SD
A: Linear additive, high censoring	0.03335	0.02167	0.02151	0.007138
B: Interactions, moderate censoring	0.6298	0.01609	0.6498	0.01904

CErr_LR	CErr_LR_SD	CErr_RSF	CErr_RSF_SD	Events_mean	Events_SD
0.7152	0.05319	0.6474	0.06304	100	9
0.7046	0.008647	0.7257	0.008396	2074	27